

JAVA

Lecture VI – I/O

I/O

- I/O system in Java:



I/O

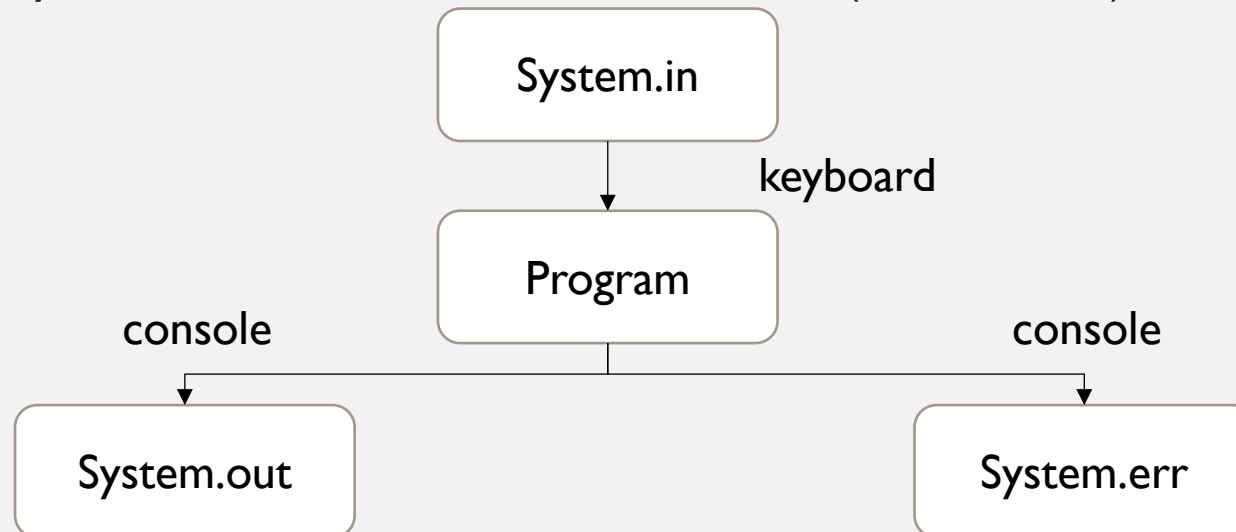
- Stream: an abstraction that either produces or consumes information
 - Linked with physical devices
 - Same I/O classes can be applied to different devices
 - Java's I/O classes are defined in the **java.io** package
- Two types of streams:
 - Byte streams: handling binary data. (Useful for files)
 - Character streams: handling characters. (more efficient)

BYTE AND CHARACTER STREAM

- Two classes of Byte Stream
 - InputStream: defines characteristics to byte input streams.
 - OutputStream: describes the behaviour of byte output streams.
- Two classes of Character Stream
 - Reader: for input
 - Writer: for output

PREDEFINED STREAMS

- Three predefined stream variables (all declared as **public**, **final**, and **static**):
 - `System.in`: the standard input stream (the keyboard).
 - `System.out`: the standard output stream (the console).
 - `System.err`: the standard error stream (the console).



READING CONSOLE INPUT

- Both Byte Stream and Character Stream can be used to perform console input.
- We will learn Character-based Stream based on **BufferedReader**.
- Can we do

```
BufferedReader br = new BufferedReader(System.in);
```

READING CONSOLE INPUT

- Both Byte Stream and Character Stream can be used to perform console input.
- We will learn Character-based Stream based on **BufferedReader**.
- Can we do

```
BufferedReader br = new BufferedReader(System.in);
```

- **No, System.in is actually a byte stream.**

READING CONSOLE INPUT

- Both Byte Stream and Character Stream can be used to perform console input.
- We will learn Character-based Stream based on **BufferedReader**.

- Can we do

```
BufferedReader br = new BufferedReader(System.in);
```

- No, `System.in` is actually a byte stream.

```
InputStreamReader sr = new InputStreamReader(System.in);
```

- It convert a byte stream to a character stream (a subclass of `Reader`).

READING CONSOLE INPUT

- Both Byte Stream and Character Stream can be used to perform console input.
- We will learn Character-based Stream based on **BufferedReader**.

- Can we do

```
BufferedReader br = new BufferedReader(System.in);
```

- No, `System.in` is actually a byte stream.

```
InputStreamReader sr = new InputStreamReader(System.in);
```

- It convert a byte stream to a character stream (a subclass of `Reader`).
- ```
BufferedReader br = new BufferedReader(sr);
```
- Now `br` is a character stream linked to the console through `System.in`

# READING CHARACTERS

- To read a single character:
- `int read()` throws `IOException`
- Be careful there is a checked exception to handle here.
- `read()` reads a single character from the input stream.

```
BufferedReader br = new BufferedReader(
 new InputStreamReader(System.in));

char c = (char)br.read(); // read() returns an integer
```

# READING STRINGS

- To read a string from the keyboard:
- `String readLine()` throws `IOException`
- It returns a `String` object
- `readLine()` reads a line of input from the input stream.

```
do{
 str = br.readLine();
 System.out.println(str);
}while(!str.equals("stop"));
```

# SCANNER

- Scanner is a class in java.util used for obtaining the input of the primitive types.
- It is much easier to use compared to BufferedReader, but slower.

```
Scanner sr = new Scanner(System.in);
```

- To read a String:

```
sr.nextLine();
```

- To read other primitive types:

```
nextInt(), nextFloat(), nextDouble(), nextShort(), ...
```

- An example of Scanner

# FILE I/O USING CHARACTER STREAM

- **FileReader**: create a reader you can use to read the content of a file.

`FileReader(String fileName)` throws `FileNotFoundException`

- `fileName` is the name of a file (could be the path to this file).
- This class throws an exception if the file does not exist.
- We need to create an object of `BufferedReader` using the `FileReader` as the input stream.
- Also need to handle the potential exceptions.

# TRY-WITH-RESOURCE

- When `FileReader` is used to access a file, we need to close it at the end.

```
FileReader fr = null; BufferedReader br = null;
try{
 fr = new FileReader("1.txt");
 br= new BufferedReader(fr);
 br.readLine();
}catch(IOException e){

}finally{
 try{
 fr.close();
 }catch(IOException e){

 }
}
```

# TRY-WITH-RESOURCE

try(declare and create resource)

**When the resource will be automatically released (closed)**

```
try(BufferedReader br = new BufferedReader(new FileReader("1.txt"))) {
 br.nextLine();
} catch (FileNotFoundException e) {
 // do something
}
```

## READ LINES FROM THE FILE

```
public static void main(String[] args){
 try(BufferedReader br = new BufferedReader(new FileReader("2.txt"))){
 while(br.readLine() != null){//if it is not the end
 System.out.println(br.readLine());
 }
 }
 catch(IOException e){
 System.out.println("file not found");
 }
}
```



# SCANNER

```
public static void main(String[] args){
 try(Scanner sr = new Scanner(new FileReader("1.txt"))){
 String str;
 while(sr.hasNextLine()){ // check if it is the end of the file
 str = sr.nextLine();
 System.out.println(str);
 }
 }
 catch(IOException e){
 e.printStackTrace();
 }
}
```

# FILEWRITER

- **FileWriter**: create a writer you can use to write to a file.

`FileWriter(String fileName)` throws `IOException`

`FileWriter(String fileName, boolean append)` throws `IOException`

- `fileName` is the name of a file.
- We append the output to the end of the file, if `append` is true
- Both throws an `IOException` on failure.

## WRITE INPUT TO A FILE

```
public static void main(String[] args) {
 try(FileWriter fw = new FileWriter("1.txt")) {
 str = "new line";
 fw.write(str);
 }
 catch(IOException e) {
 e.printStackTrace();
 }
}
```